

Gin Web 快速入门

- 1. 环境准备
 - 2. 安装 Gin
 - 3. 安装依赖
 - 4. 编写路由
 - 5. 启动服务
 - 6. 测试
 - 7. 部署
 - 8. Cookie 与 Session
 - 9. 中间件
 - 10. 静态文件
 - 11. 模板
 - 12. 数据库
 - 13. 鉴权
 - 14. 日志
 - 15. 错误处理
 - 16. 性能优化
 - 17. 安全
 - 18. 总结
-

1. 环境准备

1.1. Go 安装

访问 Go 官网 <https://golang.org/dl/> 下载对应版本的安装包

安装完成后

```
go version
```

1.2. 设置 Go 环境变量

```
go env -w GOMODCACHE=/usr/local/go/mod
go env -w GOPROXY=https://goproxy.cn,direct
```

1.3. 安装 Gin

```
go get -u github.com/gin-gonic/gin
```

1.4. Gin 快速入门

1.1	1.1
1.2	1.2 Radix 1.2.1
1.3	1.3.1
1.4	1.4.1
1.5	1.5 Recovery 1.5.1 panic
JSON 1.6	1.6.1 JSON 1.6.2
1.7	1.7.1
1.8	1.8.1
1.9	1.9 JSON/XML/HTML 1.9.1

2. 1.1

2.1. 1.1 Gin 1.1

```
package main

import (
    "github.com/gin-gonic/gin"
)

func main() {
    // 创建默认的 Gin 引擎实例，包含 Logger 和 Recovery 中间件
    r := gin.Default()

    r.GET("/", func(c *gin.Context) {
        c.String(200, "Hello, Gin!")
    })

    r.GET("/ping", func(c *gin.Context) {
        c.JSON(200, gin.H{
            "message": "pong",
        })
    })

    // 运行应用，默认监听 0.0.0.0:8080
    r.Run()
}
```

1.1

- <http://localhost:8080/> → 1.1 "Hello, Gin!"
- <http://localhost:8080/ping> → 1.1 JSON {"message": "pong"}

2.2. Gin 1.1

Gin

配置

`gin.Default()` 配置 Logger Recovery 中间件

`gin.New()` 创建一个新的 Gin 引擎

`gin.Binder()` 创建一个新的 Gin 绑定器

3. 快速入门

3.1. 快速入门

Gin 支持 HTTP 方法

```
r.GET("/path", handler) // 查询
r.POST("/path", handler) // 创建
r.PUT("/path", handler) // 更新
r.DELETE("/path", handler) // 删除
r.PATCH("/path", handler) // 部分更新
r.HEAD("/path", handler) // 头部
r.OPTIONS("/path", handler) // 选项
```

3.2. 快速入门

```
// 路径参数
r.GET("/user/:id", func(c *gin.Context) {
    id := c.Param("id")
    c.JSON(200, gin.H{"id": id})
})

// 可选参数
r.GET("/user/:id/*action", func(c *gin.Context) {
    id := c.Param("id")
    action := c.Param("action") // 可以为空
    c.JSON(200, gin.H{"id": id, "action": action})
})
```

3.3. 快速入门

```
r.GET("/search", func(c *gin.Context) {
    name := c.Query("name") // 获取查询参数，等同于
    c.Request.URL.Query().Get("name")
    age := c.DefaultQuery("age", "0") // 带默认值的查询参数
    id := c.GetQuery("id") // 返回两个值，第二个是 bool 表示是否存在
    c.JSON(200, gin.H{"name": name, "age": age, "id": id})
})
```

3.4. 📝📝📝

```
r.POST("/form", func(c *gin.Context) {
    username := c.PostForm("username")
    password := c.DefaultPostForm("password", "default password")
    c.JSON(200, gin.H{"username": username, "password": password})
})
```

3.5. 📁📁

```
// 创建路由组
v1 := r.Group("/v1")
{
    v1.GET("/users", usersHandler)
    v1.GET("/posts", postsHandler)
}

// 嵌套路由组
admin := v1.Group("/admin")
{
    admin.GET("/dashboard", dashboardHandler)
}
```

3.6. 📁📁📁📁

Gin 📁📁📁📁📁📁📁📁

1. 📁📁📁📁📁 /user/profile📁
2. 📁📁📁📁📁 /user/:id📁
3. 📁📁📁📁📁 /user/*path📁

```
r.GET("/user/profile", handler1) // 优先匹配
r.GET("/user/:id", handler2)    // 次优先
r.GET("/user/*path", handler3)  // 最后匹配
```

3.7. 📁📁📁📁📁

```
// 使用 Any 处理所有方法
r.Any("/path", handler)

// 使用 NoRoute 处理未匹配的路由
r.NoRoute(func(c *gin.Context) {
    c.JSON(404, gin.H{"error": "404 Not Found"})
})
```

```
// 使用 NoMethod 处理不支持的 HTTP 方法
r.NoMethod(func(c *gin.Context) {
    c.JSON(405, gin.H{"error": "Method Not Allowed"})
})
```

4. 🔒🔒🔒🔒🔒🔒

4.1. JSON 🔒🔒

```
type User struct {
    Name string `json:"name" binding:"required"`
    Age  int   `json:"age"`
    Email string `json:"email"`
}

r.POST("/user", func(c *gin.Context) {
    var user User
    // 绑定 JSON 到结构体
    if err := c.ShouldBindJSON(&user); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }
    c.JSON(200, gin.H{"user": user})
})
```

4.2. Form 🔒🔒

```
type LoginForm struct {
    Username string `form:"username" binding:"required"`
    Password string `form:"password" binding:"required,min=6"`
}

r.POST("/login", func(c *gin.Context) {
    var form LoginForm
    if err := c.ShouldBind(&form); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }
    c.JSON(200, gin.H{"username": form.Username})
})
```

4.3. URI 🔒🔒

```
type User struct {
    ID int `uri:"id" binding:"required"`
}
```

```
Name string `uri:"name"`
}

r.GET("/user/:id/:name", func(c *gin.Context) {
    var user User
    if err := c.ShouldBindUri(&user); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }
    c.JSON(200, gin.H{"user": user})
})
```

4.4. Header 🔒🔒

```
type HeaderRequest struct {
    Authorization string `header:"Authorization" binding:"required"`
    ContentType   string `header:"Content-Type"`
}

r.GET("/header", func(c *gin.Context) {
    var req HeaderRequest
    if err := c.ShouldBindHeader(&req); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }
    c.JSON(200, gin.H{"auth": req.Authorization})
})
```

4.5. 🔒🔒🔒🔒🔒🔒

```
// 根据 Content-Type 自动选择绑定方式
type User struct {
    Name string `json:"name" form:"name" uri:"name"`
    Age  int    `json:"age" form:"age" uri:"age"`
}

// 可以同时处理 JSON/Form/Query 参数
r.GET("/multi", func(c *gin.Context) {
    var user User
    // ShouldBind 自动检测 Content-Type
    if err := c.ShouldBind(&user); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }
    c.JSON(200, user)
})
```

4.6. Must Bind vs Should Bind

必须	必须
MustBind	返回 HTTP 400 错误
ShouldBind	返回 400 错误

4.7. 验证

4.7.1. 验证用户

```
type User struct {
    Name string `json:"name" binding:"required,min=2,max=50"` // 必填，长度2-50
    Age int `json:"age" binding:"required,gte=18,lte=100"` // 年龄18-100
    Email string `json:"email" binding:"required,email"` // 必填，邮箱格式
    Phone string `json:"phone" binding:"required,numeric,len=11"` // 必填，11位数字
    URL string `json:"url" binding:"required,url"` // URL格式
    IP string `json:"ip" binding:"required,ip"` // IP地址格式
}
```

4.7.2. 验证数据

验证规则

- required: 必填
- min: 最小值
- max: 最大值
- len: 长度
- contains: 包含
- excludes: 不包含
- startswith: 以...开头
- endswith: 以...结尾

验证方法

- eq: 等于
- ne: 不等于
- gt: 大于
- gte: 大于等于
- lt: 小于
- lte: 小于等于

验证器

- **eqfield**: 验证两个字段是否相等
- **nefield**: 验证两个字段是否不相等

验证器Validator

- **email**: 验证邮箱地址
- **url**: URL 验证
- **uri**: URI 验证
- **ip**: IP 验证
- **ipv4**: IPv4 验证
- **ipv6**: IPv6 验证
- **datetime**: 验证日期时间
- **alpha**: 验证字母
- **alphanumeric**: 验证字母数字

验证器Validator

- **oneof**: 验证一个或多个值，例如 `oneof=red green blue`

4.8. 验证器Validator

4.8.1. 验证器Validator

```
package main

import (
    "net/mail"
    "reflect"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/go-playground/validator/v10"
)

// 自定义验证函数
func validateEmail(fl validator.FieldLevel) bool {
    if email, ok := fl.Field().Interface().(string); ok {
        _, err := mail.ParseAddress(email)
        return err == nil
    }
    return false
}

// 自定义时间验证（如预订日期必须大于今天）
func validateFutureDate(fl validator.FieldLevel) bool {
    if date, ok := fl.Field().Interface().(time.Time); ok {
        return date.After(time.Now())
    }
    return false
}
```



```
func main() {
    r := gin.Default()

    // 注册自定义验证器
    if v, ok := binding.Validator.Engine().(*validate.Validate); ok {
        v.RegisterValidation("email_custom", validateEmail)
        v.RegisterValidation("future_date", validateFutureDate)
    }

    r.POST("/register", func(c *gin.Context) {
        type Register struct {
            Email    string    `json:"email"
binding:"required,email_custom"`
            BookDate time.Time `json:"book_date"
binding:"required,future_date"`
        }
        var reg Register
        if err := c.ShouldBindJSON(&reg); err != nil {
            c.JSON(400, gin.H{"error": err.Error()})
            return
        }
        c.JSON(200, gin.H{"message": "success"})
    })

    r.Run()
}
```

4.8.2. 🔒🔒🔒🔒🔒🔒

```
// 跨字段验证：密码和确认密码必须一致
func validatePasswordMatch(fl validator.StructLevel) {
    user := fl.CurrentStructInterface().(User)
    if user.Password != user.ConfirmPassword {
        fl.ReportError(user.ConfirmPassword, "ConfirmPassword",
"confirm_password", "eqfield", "Password")
    }
}

// 注册结构体验证器
if v, ok := binding.Validator.Engine().(*validate.Validate); ok {
    v.RegisterStructValidation(validatePasswordMatch, User{})
}
```

4.9. 🔒🔒🔒🔒🔒🔒🔒

```
type User struct {
    Name string `json:"name" binding:"required,min=2,max=50"`
    Age  int    `json:"age" binding:"required,gte=18,lte=100"`
    Email string `json:"email" binding:"required,email"`
}
```

```

}

// 验证失败时返回结构化的错误信息
type ValidationError struct {
    Field    string `json:"field"`
    Message  string `json:"message"`
}

func getValidationErrors(err error) []ValidationError {
    var errors []ValidationError
    for _, err := range err.(validator.ValidationErrors) {
        errors = append(errors, ValidationError{
            Field:    err.Field(),
            Message:  err.Translate(trans),
        })
    }
    return errors
}

```

5. 🏠🏠🏠🏠🏠🏠

5.1. 🏠🏠🏠🏠

```

// JSON 响应
c.JSON(200, gin.H{"message": "success"})

// XML 响应
c.XML(200, gin.H{"message": "success"})

// 纯文本响应
c.String(200, "Hello %s", "World")

// HTML 响应
c.HTML(200, "index.html", gin.H{"title": "Main"})

// 文件下载
c.File("path/to/file.txt")

// 重定向
c.Redirect(http.StatusMovedPermanently, "http://example.com")

```

5.2. Context 🏠🏠🏠🏠

```

// 获取请求信息
c.Request.Method           // HTTP 方法
c.Request.URL.Path         // 请求路径
c.Request.Header.Get("X-Request-ID") // 获取 Header
c.ClientIP()               // 客户端 IP

```

```

c.RemoteIP()           // 远程 IP

// 获取参数
c.Param("key")          // 路径参数
c.Query("key")          // 查询参数
c.PostForm("key")       // 表单参数
c.GetHeader("key")      // Header 参数

// 设置响应
c.JSON(code, object)    // JSON 响应
c.XML(code, object)     // XML 响应
c.String(code, format)  // 文本响应
c.HTML(code, template, data) // HTML 响应
c.File(filepath)        // 文件响应

// 状态码
c.Status(code)          // 设置响应状态码

// 流程控制
c.Next()               // 执行下一个处理程序
c.Abort()              // 终止后续处理
c.AbortWithStatus(code) // 终止并返回状态码

// 缓存处理
c.Set(key, value)       // 设置值到 Context
c.Get(key)              // 从 Context 获取值
c.GetString(key)        // 获取字符串
c.GetInt(key)           // 获取整数

// Cookie 和 Session
c.SetCookie(name, value, maxAge, path, domain, secure, httpOnly)
c.GetCookie(name)

// 响应头
c.Header(key, value)    // 设置响应头
c.Writer.Header().Set(key, value)

```

5.3. 🔒🔒🔒🔒🔒🔒

```

// 统一响应结构
type Response struct {
    Code    int    `json:"code"`
    Message string `json:"message"`
    Data    interface{} `json:"data,omitempty"`
}

// 成功响应
func Success(c *gin.Context, data interface{}) {
    c.JSON(200, Response{
        Code:    0,
        Message: "success",
    })
}

```

```

        Data:    data,
    })
}

// 失败响应
func Error(c *gin.Context, code int, message string) {
    c.JSON(200, Response{
        Code:    code,
        Message: message,
    })
}

// 使用示例
r.GET("/user", func(c *gin.Context) {
    user := User{ID: 1, Name: "张三"}
    Success(c, user)
})

```

5.4. 🔒🔒🔒🔒

```

r.GET("/async", func(c *gin.Context) {
    // 异步处理
    cCp := c.Copy()
    go func() {
        time.Sleep(2 * time.Second)
        // 异步处理逻辑
        log.Println("Async done")
    }()
    c.JSON(200, gin.H{"message": "processing"})
})

```

5.5. 🔒🔒🔒🔒

```

r.GET("/stream", func(c *gin.Context) {
    c.Stream(func(w *gin.ResponseWriter) bool {
        w.Write([]byte("chunk 1\n"))
        time.Sleep(time.Second)
        w.Write([]byte("chunk 2\n"))
        time.Sleep(time.Second)
        w.Write([]byte("chunk 3\n"))
        return false // 返回 true 继续发送, false 结束
    })
})

```

6. 🔒🔒🔒🔒

6.1. 自定义中间件

Gin 的中间件函数类型是 `gin.HandlerFunc`，定义如下：

```
func MyMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        // 请求处理前的逻辑
        println("Before request")

        // 继续处理后续请求
        c.Next()

        // 请求处理后的逻辑
        println("After request")
    }
}
```

6.2. 全局中间件

```
r := gin.New()
r.Use(gin.Logger())           // 日志中间件
r.Use(gin.Recovery())        // 恢复中间件
r.Use(MyMiddleware())        // 自定义全局中间件
```

6.3. 路由中间件

```
// 单个路由使用中间件
r.GET("/path", MyMiddleware(), handlerFunc)

// 路由组使用中间件
admin := r.Group("/admin")
admin.Use(AdminAuthMiddleware()) {
    admin.GET("/dashboard", dashboardHandler)
}
```

6.4. Gin 内置中间件

中间件名称	功能描述
<code>gin.Logger()</code>	记录请求日志
<code>gin.Recovery()</code>	捕获并处理 panic
<code>gin.BasicAuth()</code>	HTTP 基本认证
<code>gin.Bind()</code>	绑定请求体

6.5. 日志中间件

```
// 日志中间件
func Logger() gin.HandlerFunc {
    return func(c *gin.Context) {
        start := time.Now()
        path := c.Request.URL.Path

        c.Next() // 处理请求

        latency := time.Since(start)
        status := c.Writer.Status()
        println(path, status, latency)
    }
}

// 认证中间件
func AuthRequired() gin.HandlerFunc {
    return func(c *gin.Context) {
        token := c.GetHeader("Authorization")
        if token == "" {
            c.JSON(401, gin.H{"error": "Unauthorized"})
            c.Abort()
            return
        }
        c.Next()
    }
}
```

6.6. 用户认证中间件

```
// 在中间件中设置值，后续处理函数可以获取
func SetUserInfo() gin.HandlerFunc {
    return func(c *gin.Context) {
        // 模拟用户认证
        c.Set("user_id", 1)
        c.Set("username", "admin")
        c.Next()
    }
}

// 在处理器中获取
r.GET("/profile", SetUserInfo(), func(c *gin.Context) {
    userID, _ := c.Get("user_id")
    username, _ := c.Get("username")
    c.JSON(200, gin.H{
        "user_id": userID,
        "username": username,
    })
})
```

6.7. 🔒🔒🔒🔒🔒🔒🔒

```
import (
    "github.com/gin-contrib/cors"    // CORS 跨域
    "github.com/gin-contrib/sessions" // Session 支持
    "github.com/gin-gonic/gin/binding" // 验证器
)

// CORS 配置
r.Use(cors.New(cors.Config{
    AllowOrigins:    []string{"https://example.com"},
    AllowMethods:    []string{"GET", "POST", "PUT", "DELETE"},
    AllowHeaders:    []string{"Origin", "Content-Type", "Authorization"},
    ExposeHeaders:   []string{"Content-Length"},
    AllowCredentials: true,
}))
```

6.8. 🔒🔒🔒🔒

6.8.1. 🔒🔒🔒🔒🔒

```
func main() {
    r := gin.New()

    // 中间件按照注册顺序执行
    r.Use(Middleware1()) // 第一个执行
    r.Use(Middleware2()) // 第二个执行
    r.Use(Middleware3()) // 第三个执行

    r.GET("/test", func(c *gin.Context) {
        // 请求处理
        c.JSON(200, gin.H{"message": "ok"})
    })
    // 响应返回后，中间件按相反顺序执行后续代码
}

// 执行顺序：
// Middleware1 Before -> Middleware2 Before -> Middleware3 Before ->
// Handler
// -> Middleware3 After -> Middleware2 After -> Middleware1 After
```

6.8.2. 🔒🔒🔒🔒

```
// 只对特定路由生效的中间件
func ConditionalMiddleware(condition func(*gin.Context) bool)
```

```
gin.HandlerFunc {
    return func(c *gin.Context) {
        if condition(c) {
            // 满足条件时执行中间件逻辑
            println("条件满足, 执行中间件")
        }
        c.Next()
    }
}

// 使用示例: 只对 POST 请求进行认证
r.Use(ConditionalMiddleware(func(c *gin.Context) bool {
    return c.Request.Method == "POST"
})))

// 基于路径的中间件
func PathBasedMiddleware(pathPrefix string) gin.HandlerFunc {
    return func(c *gin.Context) {
        if strings.HasPrefix(c.Request.URL.Path, pathPrefix) {
            // 执行中间件逻辑
        }
        c.Next()
    }
}
```

6.8.3. 中间件链

```
// 中间件链
type MiddlewareChain struct {
    middlewares []gin.HandlerFunc
}

func NewMiddlewareChain(middlewares ...gin.HandlerFunc) *MiddlewareChain {
    return &MiddlewareChain{middlewares: middlewares}
}

func (mc *MiddlewareChain) Then(handler gin.HandlerFunc) gin.HandlerFunc {
    return func(c *gin.Context) {
        // 按顺序执行中间件
        for _, m := range mc.middlewares {
            m(c)
            if c.IsAborted() {
                return
            }
        }
        handler(c)
    }
}

// 使用示例
chain := NewMiddlewareChain(AuthMiddleware(), LogMiddleware(),
```



```
RateLimitMiddleware())
r.GET("/protected", chain.Then(ProtectedHandler))
```

6.8.4. 🏠🏠🏠🏠

```
import (
    "context"
    "time"
)

func TimeoutMiddleware(timeout time.Duration) gin.HandlerFunc {
    return func(c *gin.Context) {
        ctx, cancel := context.WithTimeout(c.Request.Context(), timeout)
        defer cancel()

        c.Request = c.Request.WithContext(ctx)

        // 使用 channel 监听超时
        finished := make(chan struct{})
        go func() {
            c.Next()
            close(finished)
        }()

        select {
        case <-finished:
            // 正常完成
        case <-ctx.Done():
            // 超时
            c.JSON(408, gin.H{"error": "Request Timeout"})
            c.Abort()
        }
    }
}

// 使用
r.Use(TimeoutMiddleware(30 * time.Second))
```

6.8.5. 🏠🏠🏠🏠🏠🏠🏠🏠

```
import (
    "bytes"
    "io"
)

type responseWriter struct {
    gin.ResponseWriter
    body *bytes.Buffer
```

```

}

func (w responseWriter) Write(b []byte) (int, error) {
    w.body.Write(b)
    return w.ResponseWriter.Write(b)
}

func RequestLogger() gin.HandlerFunc {
    return func(c *gin.Context) {
        // 读取请求体
        var requestBody []byte
        if c.Request.Body != nil {
            requestBody, _ = io.ReadAll(c.Request.Body)
            c.Request.Body = io.NopCloser(bytes.NewBuffer(requestBody))
        }

        // 包装响应写入器
        writer := &responseWriter{
            ResponseWriter: c.Writer,
            body:            bytes.NewBuffer(nil),
        }
        c.Writer = writer

        start := time.Now()
        c.Next()
        duration := time.Since(start)

        // 记录完整日志
        log.Printf(
            "[REQUEST] %s %s | Status: %d | Duration: %v | Request: %s | Response: %s",
            c.Request.Method,
            c.Request.URL.Path,
            c.Writer.Status(),
            duration,
            string(requestBody),
            writer.body.String(),
        )
    }
}

```

7. 0000

7.1. 0000

```

import (
    "fmt"
    "net/http"
    "os"
    "path/filepath"

```

```

    "github.com/gin-gonic/gin"
)

func main() {
    r := gin.Default()
    // 确保上传目录存在
    os.MkdirAll("./uploads", 0755)

    // 单文件上传
    r.POST("/upload", func(c *gin.Context) {
        file, err := c.FormFile("file")
        if err != nil {
            c.String(http.StatusBadRequest, "获取文件失败: %s", err.Error())
            return
        }

        // 保存文件
        filename := fmt.Sprintf("%d_%s", time.Now().Unix(), file.Filename)
        if err := c.SaveUploadedFile(file, "./uploads/"+filename); err !=
nil {
            c.String(http.StatusInternalServerError, "保存文件失败: %s",
err.Error())
            return
        }

        c.JSON(200, gin.H{
            "message":    "上传成功",
            "filename":    filename,
            "size":        file.Size,
        })
    })

    // 多文件上传
    r.POST("/upload/multiple", func(c *gin.Context) {
        form, err := c.MultipartForm()
        if err != nil {
            c.String(http.StatusBadRequest, "获取表单失败: %s", err.Error())
            return
        }

        files := form.File["files"]
        var filenames []string

        for _, file := range files {
            filename := fmt.Sprintf("%d_%s", time.Now().Unix(),
file.Filename)
            if err := c.SaveUploadedFile(file, "./uploads/"+filename); err
!= nil {
                continue
            }
            filenames = append(filenames, filename)
        }
    })
}

```

```

        c.JSON(200, gin.H{"files": filenames})
    })

    r.Run()
}

```

7.2. XXXXXXXXXX

```

func uploadWithLimit() gin.HandlerFunc {
    return func(c *gin.Context) {
        // 限制最大上传大小 (10MB)
        maxSize := int64(10 << 20)
        c.Request.Body = http.MaxBytesReader(c.Writer, c.Request.Body,
maxSize)

        file, err := c.FormFile("file")
        if err != nil {
            c.String(http.StatusBadRequest, "获取文件失败")
            return
        }

        // 限制文件类型
        ext := filepath.Ext(file.Filename)
        allowedExts := map[string]bool{".jpg": true, ".png": true, ".pdf":
true}
        if !allowedExts[ext] {
            c.String(http.StatusBadRequest, "不支持的文件类型")
            return
        }

        // 打开文件用于验证
        src, err := file.Open()
        if err != nil {
            c.String(http.StatusInternalServerError, "无法打开文件")
            return
        }
        defer src.Close()

        // 读取文件头验证类型
        buffer := make([]byte, 512)
        _, err = src.Read(buffer)
        if err != nil {
            c.String(http.StatusInternalServerError, "读取文件失败")
            return
        }

        // 检查文件 MIME 类型
        contentType := http.DetectContentType(buffer)
        allowedTypes := map[string]bool{
            "image/jpeg": true,
            "image/png": true,

```

```

        "application/pdf": true,
    }
    if !allowedTypes[contentType] {
        c.String(http.StatusBadRequest, "无效的文件类型")
        return
    }

    // 保存文件
    c.SaveUploadedFile(file, "./uploads/"+file.Filename)
    c.JSON(200, gin.H{"message": "上传成功"})
}
}

```

7.3. ☒☒☒☒

```

// 直接下载文件
r.GET("/download/:filename", func(c *gin.Context) {
    filename := c.Param("filename")
    filepath := "./files/" + filename

    // 检查文件是否存在
    if _, err := os.Stat(filepath); os.IsNotExist(err) {
        c.JSON(404, gin.H{"error": "文件不存在"})
        return
    }

    // 设置下载响应头
    c.Header("Content-Description", "File Transfer")
    c.Header("Content-Transfer-Encoding", "binary")
    c.Header("Content-Disposition", "attachment; filename="+filename)
    c.Header("Content-Type", "application/octet-stream")

    c.File(filepath)
})

// 流式下载（大文件）
r.GET("/download/stream/:filename", func(c *gin.Context) {
    filename := c.Param("filename")
    filepath := "./files/" + filename

    c.FileAttachment(filepath, filename)
})

```

7.4. ☒☒☒☒☒☒

```

func main() {
    r := gin.Default()

    // 单一目录

```

```

    r.Static("/assets", "./public/assets")

    // 多个静态目录
    r.StaticFS("/static", http.Dir("/var/www/static"))
    r.StaticFS("/images", http.Dir("./images"))

    // 单文件
    r.StaticFile("/favicon.ico", "./public/favicon.ico")

    r.Run()
}

```

7.5. 📁📁📁📁📁

7.5.1. 📁📁📁📁📁

```

package main

import (
    "crypto/md5"
    "encoding/hex"
    "fmt"
    "io"
    "os"
    "path/filepath"
    "strconv"
    "sync"

    "github.com/gin-gonic/gin"
)

// 分片上传信息
type ChunkInfo struct {
    FileID      string // 文件唯一标识
    ChunkIndex  int    // 分片索引
    TotalChunks int    // 总分片数
    ChunkHash   string // 分片哈希
}

// 分片上传记录
var uploadRecords = struct {
    sync.RWMutex
    records map[string][]bool // fileID -> chunks uploaded status
}{
    records: make(map[string][]bool),
}

// 上传分片
r.POST("/upload/chunk", func(c *gin.Context) {
    fileID := c.PostForm("file_id")
    chunkIndex, _ := strconv.Atoi(c.PostForm("chunk_index"))

```

```

totalChunks, _ := strconv.Atoi(c.PostForm("total_chunks"))

// 获取分片文件
chunkFile, err := c.FormFile("chunk")
if err != nil {
    c.JSON(400, gin.H{"error": "获取分片失败"})
    return
}

// 创建临时目录
chunkDir := filepath.Join("./uploads", "chunks", fileID)
os.MkdirAll(chunkDir, 0755)

// 保存分片
chunkPath := filepath.Join(chunkDir, fmt.Sprintf("%d.chunk",
chunkIndex))
if err := c.SaveUploadedFile(chunkFile, chunkPath); err != nil {
    c.JSON(500, gin.H{"error": "保存分片失败"})
    return
}

// 更新上传记录
uploadRecords.Lock()
if _, exists := uploadRecords.records[fileID]; !exists {
    uploadRecords.records[fileID] = make([]bool, totalChunks)
}
uploadRecords.records[fileID][chunkIndex] = true
uploadedCount := countTrue(uploadRecords.records[fileID])
uploadRecords.Unlock()

c.JSON(200, gin.H{
    "message":      "分片上传成功",
    "chunk_index":  chunkIndex,
    "uploaded":     uploadedCount,
    "total_chunks": totalChunks,
})
})

// 合并分片
r.POST("/upload/merge", func(c *gin.Context) {
    var req struct {
        FileID      string `json:"file_id" binding:"required"`
        FileName    string `json:"file_name" binding:"required"`
        TotalChunks int    `json:"total_chunks" binding:"required"`
    }

    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }

    // 检查所有分片是否上传完成
    uploadRecords.RLock()
    chunks := uploadRecords.records[req.FileID]

```

```

uploadedCount := countTrue(chunks)
uploadRecords.Unlock()

if uploadedCount < req.TotalChunks {
    c.JSON(400, gin.H{
        "error":    "分片未全部上传",
        "uploaded": uploadedCount,
        "total":    req.TotalChunks,
    })
    return
}

// 合并文件
chunkDir := filepath.Join("./uploads", "chunks", req.FileID)
targetPath := filepath.Join("./uploads", req.FileName)

targetFile, err := os.Create(targetPath)
if err != nil {
    c.JSON(500, gin.H{"error": "创建文件失败"})
    return
}
defer targetFile.Close()

for i := 0; i < req.TotalChunks; i++ {
    chunkPath := filepath.Join(chunkDir, fmt.Sprintf("%d.chunk", i))
    chunkFile, err := os.Open(chunkPath)
    if err != nil {
        c.JSON(500, gin.H{"error": fmt.Sprintf("打开分片 %d 失败", i)})
        return
    }
    io.Copy(targetFile, chunkFile)
    chunkFile.Close()
    os.Remove(chunkPath) // 删除已合并的分片
}

// 清理临时目录
os.RemoveAll(chunkDir)
delete(uploadRecords.records, req.FileID)

c.JSON(200, gin.H{"message": "文件合并成功", "file_path": targetPath})
})

func countTrue(slice []bool) int {
    count := 0
    for _, v := range slice {
        if v {
            count++
        }
    }
    return count
}

```


7.5.2. 🔒🔒🔒🔒🔒🔒

```
// 文件哈希记录
var fileHashMap = struct {
    sync.RWMutex
    m map[string]string // hash -> file_path
}{
    m: make(map[string]string),
}

// 计算文件 MD5
func calculateFileMD5(filePath string) (string, error) {
    file, err := os.Open(filePath)
    if err != nil {
        return "", err
    }
    defer file.Close()

    hash := md5.New()
    if _, err := io.Copy(hash, file); err != nil {
        return "", err
    }
    return hex.EncodeToString(hash.Sum(nil)), nil
}

// 检查文件是否已存在（秒传检查）
r.POST("/upload/check", func(c *gin.Context) {
    var req struct {
        FileHash string `json:"file_hash" binding:"required"`
    }

    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }

    fileHashMap.RLock()
    filePath, exists := fileHashMap.m[req.FileHash]
    fileHashMap.RUnlock()

    if exists {
        c.JSON(200, gin.H{
            "exist":      true,
            "message":    "秒传成功",
            "file_path": filePath,
        })
        return
    }

    c.JSON(200, gin.H{"exist": false, "message": "需要上传"})
})
```

```
// 上传完成后记录哈希
r.POST("/upload/complete", func(c *gin.Context) {
    var req struct {
        FilePath string `json:"file_path" binding:"required"`
    }

    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }

    fileHash, err := calculateFileMD5(req.FilePath)
    if err != nil {
        c.JSON(500, gin.H{"error": "计算文件哈希失败"})
        return
    }

    fileHashMap.Lock()
    fileHashMap.m[fileHash] = req.FilePath
    fileHashMap.Unlock()

    c.JSON(200, gin.H{"message": "上传完成", "file_hash": fileHash})
})
```

7.5.3. 分片上传

```
// 获取已上传的分片列表
r.GET("/upload/chunks/:file_id", func(c *gin.Context) {
    fileID := c.Param("file_id")
    chunkDir := filepath.Join("./uploads", "chunks", fileID)

    uploadedChunks := []int{}
    files, err := os.ReadDir(chunkDir)
    if err == nil {
        for _, f := range files {
            if !f.IsDir() {
                var chunkIndex int
                fmt.Sscanf(f.Name(), "%d.chunk", &chunkIndex)
                uploadedChunks = append(uploadedChunks, chunkIndex)
            }
        }
    }

    c.JSON(200, gin.H{
        "file_id": fileID,
        "uploaded_chunks": uploadedChunks,
    })
})
```

8. Cookie 🗑️ Session

8.1. Cookie 🗑️🗑️

```
// 设置 Cookie
r.GET("/set-cookie", func(c *gin.Context) {
    // 基本设置
    c.SetCookie("user", "zhangsan", 3600, "/", "localhost", false, true)

    // 完整参数: name, value, maxAge, path, domain, secure, httpOnly
    c.SetCookie("token", "abc123",
        24 * 3600, // 过期时间 (秒)
        "/",      // 路径
        "example.com", // 域名
        true,      // secure - 仅 HTTPS
        true,      // httpOnly - 防止 XSS
    )
    c.JSON(200, gin.H{"message": "Cookie 设置成功"})
})

// 获取 Cookie
r.GET("/get-cookie", func(c *gin.Context) {
    // 获取单个 Cookie
    user, err := c.Cookie("user")
    if err != nil {
        c.JSON(400, gin.H{"error": "Cookie 不存在"})
        return
    }

    // 获取所有 Cookie
    cookies := c.Request.Cookies()

    c.JSON(200, gin.H{
        "user":    user,
        "cookies": cookies,
    })
})

// 删除 Cookie
r.GET("/delete-cookie", func(c *gin.Context) {
    // 设置 MaxAge 为 -1 即可删除
    c.SetCookie("user", "", -1, "/", "localhost", false, true)
    c.JSON(200, gin.H{"message": "Cookie 已删除"})
})
```

8.2. Session 🗑️🗑️

```
go get github.com/gin-contrib/sessions
go get github.com/gin-contrib/sessions/redis
```

8.2.1. 🔒🔒 Cookie 🔒 Session

```
package main

import (
    "github.com/gin-contrib/sessions"
    "github.com/gin-contrib/sessions/cookie"
    "github.com/gin-gonic/gin"
)

func main() {
    r := gin.Default()

    // 创建基于 Cookie 的 Session 存储
    store := cookie.NewStore([]byte("secret-key-123456"))

    // 配置 Session
    r.Use(sessions.Sessions("mysession", store))

    r.GET("/session/set", func(c *gin.Context) {
        session := sessions.Default(c)
        session.Set("user_id", 123)
        session.Set("username", "张三")
        session.Save() // 必须调用 Save
        c.JSON(200, gin.H{"message": "Session 设置成功"})
    })

    r.GET("/session/get", func(c *gin.Context) {
        session := sessions.Default(c)
        userID := session.Get("user_id")
        username := session.Get("username")
        c.JSON(200, gin.H{
            "user_id": userID,
            "username": username,
        })
    })

    r.GET("/session/delete", func(c *gin.Context) {
        session := sessions.Default(c)
        session.Delete("user_id")
        session.Save()
        c.JSON(200, gin.H{"message": "Session 删除成功"})
    })

    r.GET("/session/clear", func(c *gin.Context) {
        session := sessions.Default(c)
        session.Clear()
        session.Save()
        c.JSON(200, gin.H{"message": "Session 清空成功"})
    })
}
```

```
r.Run()
}
```

8.2.2. Redis Session

```
package main

import (
    "github.com/gin-contrib/sessions"
    "github.com/gin-contrib/sessions/redis"
    "github.com/gin-gonic/gin"
)

func main() {
    r := gin.Default()

    // 创建 Redis Session 存储
    store, _ := redis.NewStore(10, "tcp", "localhost:6379", "",
[]byte("secret-key"))

    // 配置 Session 选项
    store.Options(sessions.Options{
        MaxAge: 3600 * 24, // 24 小时过期
        Path: "/",
        Secure: true,
        HttpOnly: true,
    })

    r.Use(sessions.Sessions("redis-session", store))

    r.GET("/login", func(c *gin.Context) {
        session := sessions.Default(c)
        session.Set("user_id", 1)
        session.Set("role", "admin")
        session.Save()
        c.JSON(200, gin.H{"message": "登录成功"})
    })

    // 认证中间件
    authRequired := func(c *gin.Context) {
        session := sessions.Default(c)
        userID := session.Get("user_id")
        if userID == nil {
            c.JSON(401, gin.H{"error": "未登录"})
            c.Abort()
            return
        }
        c.Set("user_id", userID)
        c.Next()
    }
}
```

```

    r.GET("/profile", authRequired, func(c *gin.Context) {
        session := sessions.Default(c)
        c.JSON(200, gin.H{
            "user_id": session.Get("user_id"),
            "role":    session.Get("role"),
        })
    })

    r.Run()
}

```

8.2.3. Session 🔒🔒🔒🔒

```

// 自定义 Session 数据结构
type UserSession struct {
    UserID    uint    `json:"user_id"`
    Username  string  `json:"username"`
    Role      string  `json:"role"`
}

// 设置复杂对象到 Session
func SetUserSession(c *gin.Context, user UserSession) {
    session := sessions.Default(c)
    session.Set("user", user)
    session.Save()
}

// 获取 Session 中的用户信息
func GetUserSession(c *gin.Context) (*UserSession, bool) {
    session := sessions.Default(c)
    userInterface := session.Get("user")
    if userInterface == nil {
        return nil, false
    }

    user, ok := userInterface.(UserSession)
    if !ok {
        return nil, false
    }
    return &user, true
}

// Session 刷新（延长过期时间）
func RefreshSession(c *gin.Context) {
    session := sessions.Default(c)
    session.Options(sessions.Options{
        MaxAge: 3600 * 24, // 重置过期时间
    })
    session.Save()
}

```

9. GORM 简介

9.1. GORM 安装

```
go get -u gorm.io/gorm
go get -u gorm.io/driver/mysql
```

9.2. GORM 使用

```
package main

import (
    "log"

    "gorm.io/driver/mysql"
    "gorm.io/gorm"
)

type User struct {
    ID      uint   `gorm:"primaryKey"`
    Name    string `gorm:"size:100;not null"`
    Email   string `gorm:"uniqueIndex;size:100"`
    Age     int
}

func main() {
    // 连接数据库
    dsn := "user:password@tcp(127.0.0.1:3306)/database_name?
charset=utf8mb4&parseTime=True&loc=Local"
    db, err := gorm.Open(mysql.Open(dsn), &gorm.Config{})
    if err != nil {
        log.Panicln("连接数据库失败:", err)
    }

    // 自动迁移表结构
    db.AutoMigrate(&User{})

    // 创建记录
    user := User{Name: "张三", Email: "zhangsan@example.com", Age: 25}
    db.Create(&user)

    // 查询记录
    var foundUser User
    db.First(&foundUser, 1) // 根据 ID 查询
    // 或
    db.Where("name = ?", "张三").First(&foundUser)

    // 更新记录
    db.Model(&foundUser).Update("age", 26)
```

```
// 删除记录
db.Delete(&foundUser)
}
```

9.3. Gin 与 GORM

```
package main

import (
    "github.com/gin-gonic/gin"
    "gorm.io/driver/mysql"
    "gorm.io/gorm"
)

var db *gorm.DB

func main() {
    // 初始化数据库
    dsn := "user:password@tcp(127.0.0.1:3306)/gin_demo?
charset=utf8mb4&parseTime=True&loc=Local"
    var err error
    db, err = gorm.Open(mysql.Open(dsn), &gorm.Config{})
    if err != nil {
        panic(err)
    }

    // 自动迁移
    db.AutoMigrate(&User{})

    // 启动 Gin
    r := gin.Default()

    // 注册路由
    r.POST("/users", CreateUser)
    r.GET("/users", GetUsers)
    r.GET("/users/:id", GetUser)
    r.PUT("/users/:id", UpdateUser)
    r.DELETE("/users/:id", DeleteUser)

    r.Run()
}

// 控制器函数
func CreateUser(c *gin.Context) {
    var user User
    if err := c.ShouldBindJSON(&user); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }
    db.Create(&user)
```



```

    c.JSON(200, user)
}

func GetUsers(c *gin.Context) {
    var users []User
    db.Find(&users)
    c.JSON(200, users)
}

func GetUser(c *gin.Context) {
    id := c.Param("id")
    var user User
    if err := db.First(&user, id).Error; err != nil {
        c.JSON(404, gin.H{"error": "User not found"})
        return
    }
    c.JSON(200, user)
}

func UpdateUser(c *gin.Context) {
    id := c.Param("id")
    var user User
    if err := db.First(&user, id).Error; err != nil {
        c.JSON(404, gin.H{"error": "User not found"})
        return
    }

    var input User
    if err := c.ShouldBindJSON(&input); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }

    db.Model(&user).Updates(input)
    c.JSON(200, user)
}

func DeleteUser(c *gin.Context) {
    id := c.Param("id")
    var user User
    if err := db.First(&user, id).Error; err != nil {
        c.JSON(404, gin.H{"error": "User not found"})
        return
    }

    db.Delete(&user)
    c.JSON(200, gin.H{"message": "User deleted"})
}

```

9.4. GORM XXXX

9.4.1. XXXX

```
// 一对一关系
type User struct {
    ID      uint
    Name    string
    Profile Profile
}

type Profile struct {
    ID      uint
    UserID  uint
    Bio     string
}

// 一对多关系
type User struct {
    ID      uint
    Name    string
    Posts   []Post
}

type Post struct {
    ID      uint
    UserID  uint
    Title   string
    Body    string
}

// 多对多关系
type Student struct {
    ID      uint
    Name    string
    Courses []Course `gorm:"many2many:student_courses"`
}

type Course struct {
    ID      uint
    Name    string
}
```

9.4.2. 数据库

```
type User struct {
    ID      uint
    Name    string
}

// 创建前钩子
func (u *User) BeforeCreate(tx *gorm.DB) error {
    // 加密密码、设置默认值等
    u.Name = "user_" + u.Name
}
```

```

        return nil
    }

    // 创建后钩子
    func (u *User) AfterCreate(tx *gorm.DB) error {
        // 记录日志、发送通知等
        return nil
    }

    // 更新前钩子
    func (u *User) BeforeUpdate(tx *gorm.DB) error {
        // 验证更新权限等
        return nil
    }
}

```

9.4.3. 事务

```

func CreateUserWithPosts(db *gorm.DB, user User, posts []Post) error {
    return db.Transaction(func(tx *gorm.DB) error {
        if err := tx.Create(&user).Error; err != nil {
            return err
        }
        for i := range posts {
            posts[i].UserID = user.ID
        }
        if err := tx.Create(&posts).Error; err != nil {
            return err
        }
        return nil
    })
}

```

9.4.4. 读写分离

```

package main

import (
    "gorm.io/driver/mysql"
    "gorm.io/gorm"
)

// 读写分离配置
func setupDB() *gorm.DB {
    // 主库（写）
    masterDSN := "root:password@tcp(master-host:3306)/mydb?
charset=utf8mb4&parseTime=True"

    // 从库（读）
}

```

```

    replicaDSNs := []string{
        "root:password@tcp(replica1-host:3306)/mydb?
charset=utf8mb4&parseTime=True",
        "root:password@tcp(replica2-host:3306)/mydb?
charset=utf8mb4&parseTime=True",
    }

    db, err := gorm.Open(mysql.Open(masterDSN), &gorm.Config{})
    if err != nil {
        panic(err)
    }

    // 配置读写分离
    db.Use(NewReadWritePlugin(masterDSN, replicaDSNs))

    return db
}

// 自定义读写分离插件
type ReadWritePlugin struct {
    master *gorm.DB
    replica *gorm.DB
}

func NewReadWritePlugin(masterDSN string, replicaDSNs []string)
*ReadWritePlugin {
    master, _ := gorm.Open(mysql.Open(masterDSN))
    // 简单轮询选择从库
    replica, _ := gorm.Open(mysql.Open(replicaDSNs[0]))
    return &ReadWritePlugin{master: master, replica: replica}
}

func (p *ReadWritePlugin) Name() string {
    return "read_write_plugin"
}

func (p *ReadWritePlugin) Initialize(db *gorm.DB) error {
    return nil
}

```

9.4.5. 软删除

```

// 软删除模型
type User struct {
    ID          uint          `gorm:"primaryKey"`
    Name        string         `gorm:"size:100"`
    Email       string         `gorm:"uniqueIndex"`
    DeletedAt   gorm.DeletedAt `gorm:"index" // 软删除字段`
}

// 软删除操作

```

```

func softDeleteExample(db *gorm.DB) {
    // 软删除 - 设置 deleted_at
    db.Delete(&User{}, 1)

    // 查询时会自动过滤已删除记录
    var users []User
    db.Find(&users) // WHERE deleted_at IS NULL

    // 包含软删除记录
    db.Unscoped().Find(&users)

    // 永久删除
    db.Unscoped().Delete(&User{}, 1)

    // 恢复软删除
    db.Unscoped().Model(&User{}).Where("id = ?", 1).Update("deleted_at",
nil)
}

```

9.4.6. 乐观锁

```

// 乐观锁模型
type Product struct {
    ID      uint `gorm:"primaryKey"`
    Name    string
    Stock   int
    Version int `gorm:"version"` // 版本号字段
}

// 乐观锁更新
func updateWithOptimisticLock(db *gorm.DB, id uint, newStock int) error {
    var product Product
    if err := db.First(&product, id).Error; err != nil {
        return err
    }

    // 更新时检查版本号
    result := db.Model(&Product{}).
        Where("id = ? AND version = ?", id, product.Version).
        Updates(map[string]interface{}{
            "stock":    newStock,
            "version":  product.Version + 1,
        })

    if result.RowsAffected == 0 {
        return errors.New("并发更新冲突，请重试")
    }
    return nil
}

```

```
// 使用 GORM 原生乐观锁支持（需要插件）
// go get gorm.io/plugin/optimisticlock
```

9.4.7. 分页

```
// 分页结构
type Pagination struct {
    Page      int `form:"page" json:"page"`
    PageSize  int `form:"page_size" json:"page_size"`
    Total     int64 `json:"total"`
}

func (p *Pagination) GetOffset() int {
    return (p.Page - 1) * p.PageSize
}

func (p *Pagination) GetLimit() int {
    return p.PageSize
}

// 分页查询函数
func Paginate(db *gorm.DB, model interface{}, pagination *Pagination) error {
    var total int64
    db.Model(model).Count(&total)
    pagination.Total = total

    return db.Offset(pagination.GetOffset()).
        Limit(pagination.GetLimit()).
        Find(model).Error
}

// 使用示例
r.GET("/users", func(c *gin.Context) {
    var pagination Pagination
    if err := c.ShouldBindQuery(&pagination); err != nil {
        pagination = Pagination{Page: 1, PageSize: 10}
    }
    if pagination.Page <= 0 {
        pagination.Page = 1
    }
    if pagination.PageSize <= 0 || pagination.PageSize > 100 {
        pagination.PageSize = 10
    }

    var users []User
    if err := Paginate(db, &users, &pagination); err != nil {
        c.JSON(500, gin.H{"error": err.Error()})
        return
    }
})
```

```
c.JSON(200, gin.H{
    "data":      users,
    "pagination": pagination,
})
})
```

9.4.8. 📄 SQL 📄

```
// 原生查询
func rawSQLExample(db *gorm.DB) {
    // 查询多行
    type Result struct {
        ID    uint
        Name  string
    }
    var results []Result
    db.Raw("SELECT id, name FROM users WHERE age > ?", 18).Scan(&results)

    // 查询单行
    var user User
    db.Raw("SELECT * FROM users WHERE id = ?", 1).Scan(&user)

    // 执行原生 SQL
    db.Exec("UPDATE users SET name = ? WHERE id = ?", "张三", 1)

    // 使用命名参数
    db.Raw("SELECT * FROM users WHERE name = @name AND age > @age",
        sql.Named("name", "张三"),
        sql.Named("age", 18),
    ).Scan(&results)
}
```

10. 📄📄📄📄

10.1. CSRF 📄📄

```
go get github.com/utrack/gin-csrf
```

```
package main

import (
    "github.com/gin-gonic/gin"
    "github.com/utrack/gin-csrf"
)
```

```
func main() {
    r := gin.Default()

    // CSRF 中间件
    r.Use(csrf.Middleware(csrf.Options{
        Secret: "csrf-secret-key-123456",
        ErrorFunc: func(c *gin.Context) {
            c.JSON(403, gin.H{"error": "CSRF token invalid"})
            c.Abort()
        },
    }))

    // 获取 CSRF Token
    r.GET("/csrf-token", func(c *gin.Context) {
        token := csrf.GetToken(c)
        c.JSON(200, gin.H{"csrf_token": token})
    })

    // 表单页面 (需要包含 CSRF Token)
    r.GET("/form", func(c *gin.Context) {
        token := csrf.GetToken(c)
        c.HTML(200, "form.html", gin.H{"csrf_token": token})
    })

    // 提交表单 (自动验证 CSRF)
    r.POST("/submit", func(c *gin.Context) {
        c.JSON(200, gin.H{"message": "提交成功"})
    })

    r.Run()
}
```

🔒🔒🔒🔒🔒:

```
<form action="/submit" method="POST">
    <input type="hidden" name="_csrf" value="{{ .csrf_token }}">
    <!-- 其他表单字段 -->
    <button type="submit">提交</button>
</form>

<!-- 或在 AJAX 中使用 -->
<script>
fetch('/submit', {
    method: 'POST',
    headers: {
        'X-CSRF-Token': document.querySelector('meta[name="csrf-token"]').content
    },
    body: JSON.stringify(data)
});
</script>
```


10.2. XSS XX

```
package main

import (
    "html"
    "regexp"
    "strings"

    "github.com/gin-gonic/gin"
)

// XSS 过滤中间件
func XSSMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        // 过滤查询参数
        for key, values := range c.Request.URL.Query() {
            for i, value := range values {
                values[i] = sanitizeInput(value)
            }
            c.Request.URL.Query()[key] = values
        }
        c.Next()
    }
}

// 输入消毒函数
func sanitizeInput(input string) string {
    // HTML 转义
    input = html.EscapeString(input)

    // 移除危险字符
    input = strings.ReplaceAll(input, "<", "&lt;")
    input = strings.ReplaceAll(input, ">", "&gt;")
    input = strings.ReplaceAll(input, "\"", "&quot;")
    input = strings.ReplaceAll(input, "'", "&#x27;")

    return input
}

// 使用 bluemonday 进行更严格的 XSS 过滤
// go get github.com/microcosm-cc/bluemonday

import "github.com/microcosm-cc/bluemonday"

func sanitizeWithPolicy(input string) string {
    // 严格策略：只允许基本标签
    policy := bluemonday.StrictPolicy()
    return policy.Sanitize(input)
}
```

```
// UGC 策略：允许安全标签
func sanitizeUGCPolicy(input string) string {
    policy := bluemonday.UGCPolicy()
    return policy.Sanitize(input)
}

// 自定义策略
func customPolicy() *bluemonday.Policy {
    p := bluemonday.NewPolicy()

    // 允许的标签
    p.AllowElements("b", "i", "u", "strong", "em", "p", "br")

    // 允许的属性
    p.AllowAttrs("href").OnElements("a")

    // 强制链接使用 rel="nofollow"
    p.RequireNoFollowOnLinks(true)

    return p
}

// 使用示例
r.POST("/comment", func(c *gin.Context) {
    var req struct {
        Content string `json:"content"`
    }
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }

    // 清洗用户输入
    safeContent := sanitizeUGCPolicy(req.Content)

    // 存储到数据库
    // db.Create(&Comment{Content: safeContent})

    c.JSON(200, gin.H{"message": "评论成功", "content": safeContent})
})
```

10.3. SQL 注入

```
import (
    "database/sql"
    "strings"

    "gorm.io/gorm"
)

// 危险：直接拼接 SQL（不要这样做！）
```

```
func dangerousQuery(db *gorm.DB, name string) {
    var users []User
    db.Raw("SELECT * FROM users WHERE name = '" + name + "'").Scan(&users)
    // SQL 注入漏洞！
}

// 安全：使用参数化查询
func safeQuery(db *gorm.DB, name string) {
    var users []User
    db.Where("name = ?", name).Find(&users) // 安全
}

// 安全：原生 SQL 使用占位符
func safeRawQuery(db *gorm.DB, name string) {
    var users []User
    db.Raw("SELECT * FROM users WHERE name = ?", name).Scan(&users) // 安全
}

// 输入验证中间件
func ValidateInputMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        // 检查常见 SQL 注入模式
        suspiciousPatterns := []string{
            "';", "--", "/*", "*/", "xp_", "exec(", "execute(",
            "union select", "insert into", "delete from", "drop table",
        }

        body, _ := c.GetRawData()
        bodyStr := strings.ToLower(string(body))

        for _, pattern := range suspiciousPatterns {
            if strings.Contains(bodyStr, pattern) {
                c.JSON(400, gin.H{"error": "可疑的输入"})
                c.Abort()
                return
            }
        }

        // 恢复请求体
        c.Request.Body = io.NopCloser(bytes.NewBuffer(body))
        c.Next()
    }
}

// 白名单验证
func validateIdentifier(identifier string) bool {
    // 只允许字母、数字、下划线
    matched, _ := regexp.MatchString("^[a-zA-Z0-9_]+$", identifier)
    return matched
}

// 安全的动态表名查询
func safeDynamicQuery(db *gorm.DB, tableName string, id uint) error {
    // 验证表名
```

```

    allowedTables := map[string]bool{
        "users":    true,
        "products": true,
        "orders":   true,
    }

    if !allowedTables[tableName] {
        return errors.New("invalid table name")
    }

    var result interface{}
    return db.Table(tableName).Where("id = ?", id).First(&result).Error
}

```

10.4. 安装依赖

```
go get golang.org/x/crypto/bcrypt
```

```

package main

import (
    "golang.org/x/crypto/bcrypt"
)

// 密码加密
func HashPassword(password string) (string, error) {
    // cost 越高越安全，但耗时越长（默认 10，推荐 12-14）
    bytes, err := bcrypt.GenerateFromPassword([]byte(password),
        bcrypt.DefaultCost)
    return string(bytes), err
}

// 密码验证
func CheckPassword(password, hash string) bool {
    err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
    return err == nil
}

// 注册接口
r.POST("/register", func(c *gin.Context) {
    var req struct {
        Username string `json:"username" binding:"required,min=3,max=50"`
        Password string `json:"password" binding:"required,min=8,max=72"`
    }
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }
}

```

```
// 密码强度检查
if !isStrongPassword(req.Password) {
    c.JSON(400, gin.H{"error": "密码强度不足，需要包含大小写字母、数字和特殊字
符"})
    return
}

// 加密密码
hashedPassword, err := HashPassword(req.Password)
if err != nil {
    c.JSON(500, gin.H{"error": "密码加密失败"})
    return
}

// 存储用户
user := User{
    Username: req.Username,
    Password: hashedPassword,
}
db.Create(&user)

c.JSON(200, gin.H{"message": "注册成功"})
})

// 密码强度检查
func isStrongPassword(password string) bool {
    var hasUpper, hasLower, hasNumber, hasSpecial bool
    for _, c := range password {
        switch {
        case unicode.IsUpper(c):
            hasUpper = true
        case unicode.IsLower(c):
            hasLower = true
        case unicode.IsNumber(c):
            hasNumber = true
        case unicode.IsPunct(c) || unicode.IsSymbol(c):
            hasSpecial = true
        }
    }
    return hasUpper && hasLower && hasNumber && hasSpecial
}

// 登录接口
r.POST("/login", func(c *gin.Context) {
    var req struct {
        Username string `json:"username" binding:"required"`
        Password string `json:"password" binding:"required"`
    }
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }

    // 查询用户
```

```

var user User
if err := db.Where("username = ?", req.Username).First(&user).Error;
err != nil {
    c.JSON(401, gin.H{"error": "用户名或密码错误"})
    return
}

// 验证密码（使用固定时间比较防止时序攻击）
if !CheckPassword(req.Password, user.Password) {
    c.JSON(401, gin.H{"error": "用户名或密码错误"})
    return
}

// 生成 Token
token, _ := GenerateToken(user.ID, user.Username)
c.JSON(200, gin.H{"token": token})
})

```

10.5. 🔒🔒🔒🔒🔒

```

// 安全头中间件
func SecurityHeadersMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        // 防止点击劫持
        c.Header("X-Frame-Options", "DENY")

        // 防止 MIME 类型嗅探
        c.Header("X-Content-Type-Options", "nosniff")

        // XSS 保护
        c.Header("X-XSS-Protection", "1; mode=block")

        // 内容安全策略
        c.Header("Content-Security-Policy", "default-src 'self'; script-src
'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'")

        // HTTPS 强制（生产环境启用）
        c.Header("Strict-Transport-Security", "max-age=31536000;
includeSubDomains")

        // 禁用缓存敏感页面
        c.Header("Cache-Control", "no-store, no-cache, must-revalidate,
proxy-revalidate")
        c.Header("Pragma", "no-cache")
        c.Header("Expires", "0")

        // 隐藏服务器信息
        c.Header("X-Powered-By", "")

        c.Next()
    }
}

```

```
}

// 使用
r.Use(SecurityHeadersMiddleware())
```

10.6. 🔒🔒🔒🔒🔒🔒

```
// 日志脱敏
func MaskSensitiveData(data string) string {
    if len(data) <= 4 {
        return "****"
    }
    return data[:2] + strings.Repeat("*", len(data)-4) + data[len(data)-2:]
}

// 日志中间件（脱敏版）
func SensitiveLogMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        start := time.Now()

        // 记录请求（脱敏）
        log.Printf("[REQUEST] %s %s", c.Request.Method, c.Request.URL.Path)

        // 不要记录敏感 Header
        auth := c.GetHeader("Authorization")
        if auth != "" {
            log.Printf("[AUTH] Token: %s...", auth[:min(10, len(auth))])
        }

        c.Next()

        log.Printf("[RESPONSE] %d %v", c.Writer.Status(),
            time.Since(start))
    }
}

// 响应数据过滤
type UserResponse struct {
    ID      uint   `json:"id"`
    Username string `json:"username"`
    Email   string `json:"email"`
    // 不返回密码字段
}

func ToUserResponse(user User) UserResponse {
    return UserResponse{
        ID:      user.ID,
        Username: user.Username,
        Email:   MaskEmail(user.Email),
    }
}
```

```
func MaskEmail(email string) string {
    parts := strings.Split(email, "@")
    if len(parts) != 2 {
        return email
    }
    name := parts[0]
    if len(name) <= 2 {
        return name[:1] + "***@" + parts[1]
    }
    return name[:2] + "***@" + parts[1]
}
```

11. 🔒🔒

11.1. 🔒🔒🔒🔒

```
package main

import (
    "net/http"
    "net/http/httptest"
    "testing"

    "github.com/gin-gonic/gin"
    "github.com/stretchr/testify/assert"
)

func setupRouter() *gin.Engine {
    gin.SetMode(gin.TestMode)
    r := gin.New()

    r.GET("/ping", func(c *gin.Context) {
        c.JSON(200, gin.H{"message": "pong"})
    })

    r.GET("/hello/:name", func(c *gin.Context) {
        name := c.Param("name")
        c.JSON(200, gin.H{"greeting": "Hello, " + name})
    })

    return r
}

func TestPing(t *testing.T) {
    r := setupRouter()

    w := httptest.NewRecorder()
    req, _ := http.NewRequest("GET", "/ping", nil)
    r.ServeHTTP(w, req)
```



```

    assert.Equal(t, 200, w.Code)
    assert.Contains(t, w.Body.String(), "pong")
}

func TestHello(t *testing.T) {
    r := setupRouter()

    w := httptest.NewRecorder()
    req, _ := http.NewRequest("GET", "/hello/World", nil)
    r.ServeHTTP(w, req)

    assert.Equal(t, 200, w.Code)
    assert.Contains(t, w.Body.String(), "Hello, World")
}

```

11.2. 测试路由

```

func TestGetUser(t *testing.T) {
    r := setupRouter()
    r.GET("/user/:id", func(c *gin.Context) {
        id := c.Param("id")
        c.JSON(200, gin.H{"id": id})
    })

    w := httptest.NewRecorder()
    req, _ := http.NewRequest("GET", "/user/123", nil)
    r.ServeHTTP(w, req)

    assert.Equal(t, 200, w.Code)
    assert.Contains(t, w.Body.String(), "123")
}

```

11.3. 测试JSON

```

func TestCreateUser(t *testing.T) {
    r := setupRouter()
    r.POST("/user", func(c *gin.Context) {
        var input struct {
            Name string `json:"name"`
            Age  int    `json:"age"`
        }
        if err := c.ShouldBindJSON(&input); err != nil {
            c.JSON(400, gin.H{"error": err.Error()})
            return
        }
        c.JSON(201, gin.H{
            "name": input.Name,
            "age":  input.Age,
        })
    })
}

```

```

    })
})

body := `{"name":"张三","age":25}`
w := httptest.NewRecorder()
req, _ := http.NewRequest("POST", "/user", strings.NewReader(body))
req.Header.Set("Content-Type", "application/json")
r.ServeHTTP(w, req)

assert.Equal(t, 201, w.Code)
assert.Contains(t, w.Body.String(), "张三")
}

```

11.4. 测试数据库

```

import (
    "os"
    "testing"

    "gorm.io/driver/sqlite"
    "gorm.io/gorm"
)

var db *gorm.DB

func TestMain(m *testing.M) {
    // 设置测试数据库
    dsn := "file::memory:?cache=shared"
    var err error
    db, err = gorm.Open(sqlite.Open(dsn), &gorm.Config{})
    if err != nil {
        panic(err)
    }

    // 运行测试
    os.Exit(m.Run())
}

func TestDatabase(t *testing.T) {
    // 自动迁移
    db.AutoMigrate(&User{})

    // 测试创建
    user := User{Name: "Test"}
    db.Create(&user)
    assert.NotZero(t, user.ID)

    // 测试查询
    var found User
    db.First(&found, user.ID)
}

```

```
    assert.Equal(t, "Test", found.Name)
}
```

11.5. testify 测试

```
go get github.com/stretchr/testify
```

```
import (
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestExample(t *testing.T) {
    // 断言
    assert.Equal(t, expected, actual)
    assert.NotNil(t, object)
    assert.True(t, condition)
    assert.Contains(t, slice, element)

    // 需要, 否则测试失败
    require.Equal(t, expected, actual)
    require.NotNil(t, object)
}
```

12. gin-gonic

12.1. Basic 测试

```
import (
    "crypto/sha256"
    "encoding/base64"

    "github.com/gin-gonic/gin"
)

func main() {
    r := gin.Default()

    // 简单 Basic Auth
    r.Use(gin.BasicAuth(gin.Accounts{
        "admin": "secret",
    }))

    r.GET("/protected", func(c *gin.Context) {
        user, password, hasAuth := c.Request.BasicAuth()
        if hasAuth {
```

```

        c.JSON(200, gin.H{
            "user":    user,
            "password": password,
        })
    }
})

r.Run()
}

```

12.2. JWT XX

```
go get github.com/golang-jwt/jwt/v5
```

```

import (
    "fmt"
    "net/http"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/golang-jwt/jwt/v5"
)

// JWT 密钥
var jwtKey = []byte("your-secret-key")

type Claims struct {
    UserID    uint   `json:"user_id"`
    Username  string `json:"username"`
    jwt.RegisteredClaims
}

// 生成 JWT
func GenerateToken(userID uint, username string) (string, error) {
    claims := Claims{
        UserID:    userID,
        Username:  username,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(24 * time.Hour)),
            IssuedAt:  jwt.NewNumericDate(time.Now()),
        },
    }

    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString(jwtKey)
}

// JWT 中间件
func AuthMiddleware() gin.HandlerFunc {

```

```

    return func(c *gin.Context) {
        tokenStr := c.GetHeader("Authorization")
        if tokenStr == "" {
            c.JSON(http.StatusUnauthorized, gin.H{"error": "Token
required"})
            c.Abort()
            return
        }

        // 解析 Bearer token
        tokenStr = tokenStr[7:] // 移除 "Bearer "

        claims := &Claims{}
        token, err := jwt.ParseWithClaims(tokenStr, claims, func(token
*jwt.Token) (interface{}, error) {
            return jwtKey, nil
        })

        if err != nil || !token.Valid {
            c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid
token"})
            c.Abort()
            return
        }

        // 将用户信息存入 Context
        c.Set("user_id", claims.UserID)
        c.Set("username", claims.Username)
        c.Next()
    }
}

// 登录接口
func loginHandler(c *gin.Context) {
    var input struct {
        Username string `json:"username"`
        Password string `json:"password"`
    }
    if err := c.ShouldBindJSON(&input); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    // 验证用户（实际应该查询数据库）
    if input.Username != "admin" || input.Password != "password" {
        c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid
credentials"})
        return
    }

    token, err := GenerateToken(1, input.Username)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
        return
    }
}

```

```

    }

    c.JSON(http.StatusOK, gin.H{"token": token})
}

// 受保护的路由
func profileHandler(c *gin.Context) {
    userID, _ := c.Get("user_id")
    username, _ := c.Get("username")

    c.JSON(http.StatusOK, gin.H{
        "user_id": userID,
        "username": username,
    })
}

func main() {
    r := gin.Default()

    r.POST("/login", loginHandler)

    protected := r.Group("/api")
    protected.Use(AuthMiddleware())
    {
        protected.GET("/profile", profileHandler)
    }

    r.Run()
}

```

12.3. CORS 🔒🔒🔒

```
go get github.com/gin-contrib/cors
```

```

import (
    "github.com/gin-contrib/cors"
    "github.com/gin-gonic/gin"
)

func main() {
    r := gin.Default()

    // 简单配置
    r.Use(cors.Default())

    // 详细配置
    r.Use(cors.New(cors.Config{
        AllowOrigins: []string{"https://example.com",
            "http://localhost:3000"},
    }

```

```

        AllowMethods: []string{"GET", "POST", "PUT", "DELETE", "OPTIONS"},
        AllowHeaders: []string{"Origin", "Content-Type", "Accept",
"Authorization"},
        ExposeHeaders: []string{"Content-Length"},
        AllowCredentials: true,
        MaxAge: 12 * time.Hour,
    )))

    r.GET("/api", func(c *gin.Context) {
        c.JSON(200, gin.H{"message": "OK"})
    })

    r.Run()
}

```

12.4. 🔒🔒🔒🔒

```

import (
    "net/http"
    "sync"
    "time"

    "github.com/gin-gonic/gin"
)

// 简单的内存限流器
type RateLimiter struct {
    requests map[string][]time.Time
    mu        sync.Mutex
    limit     int
    window    time.Duration
}

func NewRateLimiter(limit int, window time.Duration) *RateLimiter {
    return &RateLimiter{
        requests: make(map[string][]time.Time),
        limit:    limit,
        window:   window,
    }
}

func (rl *RateLimiter) Allow(key string) bool {
    rl.mu.Lock()
    defer rl.mu.Unlock()

    now := time.Now()
    windowStart := now.Add(-rl.window)

    // 清理过期的请求记录
    var valid []time.Time
    for _, t := range rl.requests[key] {

```

```

        if t.After(windowStart) {
            valid = append(valid, t)
        }
    }

    if len(valid) >= rl.limit {
        rl.requests[key] = valid
        return false
    }

    rl.requests[key] = append(valid, now)
    return true
}

func RateLimitMiddleware(limiter *RateLimiter) gin.HandlerFunc {
    return func(c *gin.Context) {
        ip := c.ClientIP()
        if !limiter.Allow(ip) {
            c.JSON(http.StatusTooManyRequests, gin.H{
                "error": "Too many requests",
            })
            c.Abort()
            return
        }
        c.Next()
    }
}

func main() {
    r := gin.Default()
    limiter := NewRateLimiter(10, time.Minute)

    r.Use(RateLimitMiddleware(limiter))
    r.GET("/api", func(c *gin.Context) {
        c.JSON(200, gin.H{"message": "OK"})
    })

    r.Run()
}

```

13. 限制请求频率

13.1. Gin 限制请求频率

```

import "github.com/gin-gonic/gin"

func main() {
    // 开发模式
    gin.SetMode(gin.DebugMode)
}

```



```
// 测试模式
gin.SetMode(gin.TestMode)

// 生产模式（禁用所有调试功能，性能最高）
gin.SetMode(gin.ReleaseMode)

r := gin.Default()
r.Run()
}
```

13.2. 📁📁📁📁📁📁

```
func main() {
    // 使用 New() 而不是 Default(), 避免日志开销
    r := gin.New()

    // 仅在需要时添加日志
    r.Use(gin.Logger())

    // 添加 Recovery
    r.Use(gin.Recovery())
}
```

13.3. 📁📁📁📁

```
// 避免在处理函数中创建大对象
var bufferPool = sync.Pool{
    New: func() interface{} {
        return make([]byte, 4096)
    },
}

func handler(c *gin.Context) {
    buf := bufferPool.Get().([]byte)
    defer bufferPool.Put(buf)

    // 使用 buf 处理请求
    // ...
}
```

13.4. 📁📁📁📁📁

```
import (
    "net/http"
    "time"
)
```

```
func main() {
    r := gin.Default()

    // 配置 HTTP 服务器
    s := &http.Server{
        Addr:           ":8080",
        Handler:         r,
        ReadTimeout:    10 * time.Second,
        WriteTimeout:   10 * time.Second,
        MaxHeaderBytes: 1 << 20,
    }

    s.ListenAndServe()
}
```

13.5. 📄📄 gzip 📄📄

```
go get github.com/klauspost/compress/gzhttp
```

```
import (
    "github.com/gin-gonic/gin"
    "github.com/klauspost/compress/gzhttp"
)

func main() {
    r := gin.Default()

    // 启用 gzip 压缩
    r.Use(gzhttp.Gzip(gzhttp.DefaultCompression))

    r.GET("/api", func(c *gin.Context) {
        c.JSON(200, gin.H{"message": "Hello"})
    })

    r.Run()
}
```

14. 📄📄📄📄

14.1. 📄📄📄📄📄📄📄📄

```
mygin/
├─ main.go      # 应用入口
├─ go.mod       # Go 模块文件
└─ config/     # 配置目录
```

```
|   └─ config.go           # 配置文件
|─ database/              # 数据库目录
|   └─ database.go        # 数据库连接
|─ models/                # 模型目录
|   └─ user.go            # 用户模型
|─ controllers/           # 控制器目录
|   └─ user_controller.go # 用户控制器
|─ services/              # 服务层目录
|   └─ user_service.go    # 用户服务
|─ routes/                # 路由目录
|   └─ router.go          # 路由配置
|─ middlewares/           # 中间件目录
|   └─ auth.go            # 认证中间件
|─ handlers/              # 处理器目录
|   └─ response.go        # 响应处理
|─ utils/                 # 工具函数目录
|   └─ helpers.go         # 辅助函数
|─ static/                # 静态文件目录
|   └─ css/
|   └─ js/
|   └─ img/
└─ templates/             # 模板目录
   └─ index.html
```

14.2. 数据库配置

文件	说明
config/	数据库配置
models/	数据库模型
controllers/	数据库控制器
services/	数据库服务
routes/	数据库路由
middlewares/	数据库中间件
utils/	数据库工具函数

14.3. 路由配置

```
// routes/router.go
package routes

import (
    "mygin/controllers"
    "mygin/middlewares"

    "github.com/gin-gonic/gin"
)
```

```
func SetupRouter() *gin.Engine {
    r := gin.Default()

    // 用户相关路由
    users := r.Group("/users")
    {
        users.GET("", controllers.GetUsers)
        users.GET("/:id", controllers.GetUser)
        users.POST("", middlewares.AuthRequired(), controllers.CreateUser)
        users.PUT("/:id", middlewares.AuthRequired(),
controllers.UpdateUser)
        users.DELETE("/:id", middlewares.AuthRequired(),
controllers.DeleteUser)
    }

    return r
}
```

15. 🐼🐼🐼🐼

15.1. WebSocket 🐼🐼

```
go get github.com/gorilla/websocket
```

```
package main

import (
    "log"
    "net/http"

    "github.com/gin-gonic/gin"
    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool {
        return true // 允许所有来源，生产环境需要限制
    },
}

func handleWebSocket(c *gin.Context) {
    conn, err := upgrader.Upgrade(c.Writer, c.Request, nil)
    if err != nil {
        log.Println("WebSocket 升级失败:", err)
        return
    }
    defer conn.Close()
```

```

    for {
        // 读取消息
        messageType, message, err := conn.ReadMessage()
        if err != nil {
            log.Println("读取消息失败:", err)
            break
        }

        log.Printf("收到消息: %s", message)

        // 发送消息
        err = conn.WriteMessage(messageType, []byte("服务器收到: "+string(message)))
        if err != nil {
            log.Println("发送消息失败:", err)
            break
        }
    }
}

func main() {
    r := gin.Default()

    r.GET("/ws", handleWebSocket)

    r.Run(":8080")
}

```

15.2. Server-Sent Events (SSE)

```

package main

import (
    "time"

    "github.com/gin-gonic/gin"
)

func handleSSE(c *gin.Context) {
    // 设置 SSE 响应头
    c.Header("Content-Type", "text/event-stream")
    c.Header("Cache-Control", "no-cache")
    c.Header("Connection", "keep-alive")

    // 发送事件
    for i := 0; i < 10; i++ {
        c.SSEvent("message", gin.H{
            "id":      i,
            "time":     time.Now().Format(time.RFC3339),
            "data":     "这是第 " + string(rune(i+1)) + " 条消息",
        })
    }
}

```

```

    })
    c.Writer.Flush()
    time.Sleep(time.Second)
}
}

func main() {
    r := gin.Default()

    r.GET("/events", handleSSE)

    r.Run()
}

```

15.3. 模板引擎

15.3.1. 模板引擎

```

package main

import (
    "net/http"

    "github.com/gin-gonic/gin"
)

func main() {
    r := gin.Default()

    // 加载模板文件
    r.LoadHTMLGlob("templates/*")

    r.GET("/", func(c *gin.Context) {
        c.HTML(http.StatusOK, "index.html", gin.H{
            "title":    "主页",
            "message": "Hello, Gin!",
        })
    })

    r.Run()
}

```

templates/index.html:

```

<!DOCTYPE html>
<html>
<head>
    <title>{{ .title }}</title>
</head>

```

```
<body>
  <h1>{{ .message }}</h1>
</body>
</html>
```

15.3.2. 加载模板

```
// 加载多层目录的模板
r.LoadHTMLGlob("templates/**/*.html")

// 或加载多个目录
r.LoadHTMLFiles("templates/base.html", "templates/index.html")
```

15.3.3. 自定义模板函数

```
import (
    "html/template"
    "time"
)

func formatAsDate(t time.Time) string {
    return t.Format("2006-01-02")
}

func main() {
    r := gin.Default()

    // 设置自定义模板函数
    r.SetFuncMap(template.FuncMap{
        "formatAsDate": formatAsDate,
    })

    r.LoadHTMLGlob("templates/*.html")

    r.GET("/date", func(c *gin.Context) {
        c.HTML(http.StatusOK, "date.html", gin.H{
            "now": time.Now(),
        })
    })

    r.Run()
}
```

templates/date.html:

```
<p>当前时间: {{ .now | formatAsDate }}</p>
```

15.4. 国际化 (i18n)

```
go get github.com/nicksnyder/go-i18n/v2/i18n
go get golang.org/x/text/language
```

```
package main

import (
    "embed"
    "log"

    "github.com/BurntSushi/toml"
    "github.com/gin-gonic/gin"
    "github.com/nicksnyder/go-i18n/v2/i18n"
    "golang.org/x/text/language"
)

//go:embed locales/*
var localesFS embed.FS

func main() {
    // 初始化 i18n
    bundle := i18n.NewBundle(language.English)
    bundle.RegisterUnmarshalFunc("toml", toml.Unmarshal)
    bundle.LoadMessageFileFS(localesFS, "locales/zh-CN.toml")
    bundle.LoadMessageFileFS(localesFS, "locales/en-US.toml")

    r := gin.Default()

    r.Use(func(c *gin.Context) {
        lang := c.GetHeader("Accept-Language")
        if lang == "" {
            lang = "zh-CN"
        }

        localizer := i18n.NewLocalizer(bundle, lang)
        c.Set("localizer", localizer)
        c.Next()
    })

    r.GET("/hello", func(c *gin.Context) {
        localizer := c.MustGet("localizer").(*i18n.Localizer)

        msg, _ := localizer.Localize(&i18n.LocalizeConfig{
            MessageID: "welcome",
        })

        c.JSON(200, gin.H{"message": msg})
    })
}
```



```
r.Run()
}
```

15.5. 定时任务

```
package main

import (
    "log"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/robfig/cron/v3"
)

func main() {
    r := gin.Default()

    // 创建定时任务
    c := cron.New()

    // 每分钟执行一次
    c.AddFunc("* * * * *", func() {
        log.Println("每分钟任务执行")
    })

    // 每小时执行一次
    c.AddFunc("0 * * * *", func() {
        log.Println("每小时任务执行")
    })

    // 每天凌晨2点执行
    c.AddFunc("0 2 * * *", func() {
        log.Println("每天凌晨2点任务执行")
    })

    c.Start()
    defer c.Stop()

    r.GET("/", func(c *gin.Context) {
        c.JSON(200, gin.H{"message": "服务运行中"})
    })

    r.Run()
}
```

15.6. 定时任务

```
go get gopkg.in/gomail.v2
```

```
package main

import (
    "github.com/gin-gonic/gin"
    "gopkg.in/gomail.v2"
)

type EmailRequest struct {
    To      string `json:"to" binding:"required,email"`
    Subject string `json:"subject" binding:"required"`
    Body    string `json:"body" binding:"required"`
}

func sendEmail(to, subject, body string) error {
    m := gomail.NewMessage()
    m.SetHeader("From", "sender@example.com")
    m.SetHeader("To", to)
    m.SetHeader("Subject", subject)
    m.SetBody("text/html", body)

    d := gomail.NewDialer("smtp.example.com", 587, "username", "password")

    return d.DialAndSend(m)
}

func main() {
    r := gin.Default()

    r.POST("/send-email", func(c *gin.Context) {
        var req EmailRequest
        if err := c.ShouldBindJSON(&req); err != nil {
            c.JSON(400, gin.H{"error": err.Error()})
            return
        }

        if err := sendEmail(req.To, req.Subject, req.Body); err != nil {
            c.JSON(500, gin.H{"error": "发送失败"})
            return
        }

        c.JSON(200, gin.H{"message": "发送成功"})
    })

    r.Run()
}
```

15.7.1. XXXX

```
package main

import (
    "time"

    "github.com/gin-gonic/gin"
)

type CacheItem struct {
    Value      interface{}
    ExpiredAt  time.Time
}

var cache = make(map[string]CacheItem)

func SetCache(key string, value interface{}, duration time.Duration) {
    cache[key] = CacheItem{
        Value:      value,
        ExpiredAt:  time.Now().Add(duration),
    }
}

func GetCache(key string) (interface{}, bool) {
    item, exists := cache[key]
    if !exists || time.Now().After(item.ExpiredAt) {
        delete(cache, key)
        return nil, false
    }
    return item.Value, true
}

// 缓存中间件
func CacheMiddleware(duration time.Duration) gin.HandlerFunc {
    return func(c *gin.Context) {
        key := c.Request.URL.Path
        if val, exists := GetCache(key); exists {
            c.JSON(200, val)
            c.Abort()
            return
        }
        c.Next()
    }
}

func main() {
    r := gin.Default()

    r.GET("/api/data", CacheMiddleware(5*time.Minute), func(c *gin.Context) {
        data := gin.H{"message": "这是缓存数据"}
    })
}
```

```

        SetCache(c.Request.URL.Path, data, 5*time.Minute)
        c.JSON(200, data)
    })

    r.Run()
}

```

15.7.2. Redis ☒☒

```
go get github.com/go-redis/redis/v8
```

```

import (
    "context"
    "encoding/json"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/go-redis/redis/v8"
)

var ctx = context.Background()
var rdb *redis.Client

func initRedis() {
    rdb = redis.NewClient(&redis.Options{
        Addr:     "localhost:6379",
        Password: "", // 密码
        DB:       0,  // 数据库
    })
}

// Redis 缓存中间件
func RedisCacheMiddleware(duration time.Duration) gin.HandlerFunc {
    return func(c *gin.Context) {
        key := "cache:" + c.Request.URL.Path

        // 尝试从 Redis 获取
        val, err := rdb.Get(ctx, key).Result()
        if err == nil {
            c.Data(200, "application/json; charset=utf-8", []byte(val))
            c.Abort()
            return
        }

        c.Next()
    }
}

// 保存到 Redis

```

```
func SaveToRedis(key string, data interface{}, duration time.Duration)
error {
    json, _ := json.Marshal(data)
    return rdb.Set(ctx, key, json, duration).Err()
}

func main() {
    initRedis()
    r := gin.Default()

    r.GET("/api/cached", RedisCacheMiddleware(5*time.Minute), func(c
    *gin.Context) {
        data := gin.H{"message": "Redis 缓存数据"}
        SaveToRedis("cache:"+c.Request.URL.Path, data, 5*time.Minute)
        c.JSON(200, data)
    })

    r.Run()
}
```

15.8. 图灵社区会员

```
go get go.uber.org/zap
```

```
package main

import (
    "time"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"
)

var logger *zap.Logger

func initLogger() {
    var err error
    logger, err = zap.NewProduction()
    if err != nil {
        panic(err)
    }
}

func ZapLogger() gin.HandlerFunc {
    return func(c *gin.Context) {
        start := time.Now()
        path := c.Request.URL.Path
        query := c.Request.URL.RawQuery
```

```

        c.Next()

        end := time.Now()
        latency := end.Sub(start)

        logger.Info("HTTP Request",
            zap.Int("status", c.Writer.Status()),
            zap.String("method", c.Request.Method),
            zap.String("path", path),
            zap.String("query", query),
            zap.String("ip", c.ClientIP()),
            zap.Duration("latency", latency),
            zap.String("user-agent", c.Request.UserAgent()),
        )
    }
}

func main() {
    initLogger()
    defer logger.Sync()

    r := gin.New()
    r.Use(ZapLogger())
    r.Use(gin.Recovery())

    r.GET("/", func(c *gin.Context) {
        c.JSON(200, gin.H{"message": "OK"})
    })

    r.Run()
}

```

15.9. API 文档 (Swagger)

```

go get -u github.com/swaggo/swag/cmd/swag
go get -u github.com/swaggo/gin-swagger
go get -u github.com/swaggo/files

```

```

package main

import (
    "github.com/gin-gonic/gin"
    swaggerFiles "github.com/swaggo/files"
    ginSwagger "github.com/swaggo/gin-swagger"

    _ "myapp/docs" // docs 由 swag init 生成
)

// @title          Gin API 示例

```

```
// @version      1.0
// @description   这是一个 Gin 框架的 API 示例项目
// @termsOfService http://swagger.io/terms/

// @contact.name  API Support
// @contact.url   http://www.swagger.io/support
// @contact.email support@swagger.io

// @license.name  Apache 2.0
// @license.url   http://www.apache.org/licenses/LICENSE-2.0.html

// @host          localhost:8080
// @BasePath      /api/v1
func main() {
    r := gin.Default()

    // Swagger 文档路由
    r.GET("/swagger/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))

    // API 路由
    v1 := r.Group("/api/v1")
    {
        // @Summary      获取用户列表
        // @Description   获取所有用户的列表
        // @Tags          users
        // @Accept        json
        // @Produce       json
        // @Success       200 {array} User
        // @Router        /users [get]
        v1.GET("/users", GetUsers)

        // @Summary      创建用户
        // @Description   创建新用户
        // @Tags          users
        // @Accept        json
        // @Produce       json
        // @Param          user body      User true "用户信息"
        // @Success       201 {object} User
        // @Router        /users [post]
        v1.POST("/users", CreateUser)
    }

    r.Run()
}
```

运行命令

```
swag init
```

15.10. 生成 ID

```
package main

import (
    "github.com/gin-gonic/gin"
    "github.com/google/uuid"
)

func RequestIDMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        requestID := c.GetHeader("X-Request-ID")
        if requestID == "" {
            requestID = uuid.New().String()
        }

        c.Set("request_id", requestID)
        c.Writer.Header().Set("X-Request-ID", requestID)

        c.Next()
    }
}

func main() {
    r := gin.Default()
    r.Use(RequestIDMiddleware())

    r.GET("/", func(c *gin.Context) {
        requestID, _ := c.Get("request_id")
        c.JSON(200, gin.H{
            "message":    "OK",
            "request_id": requestID,
        })
    })

    r.Run()
}
```

16. 错误处理

16.1. 错误处理

```
// 统一错误响应
func HandleError(c *gin.Context, err error) {
    c.JSON(400, gin.H{
        "error": err.Error(),
    })
}

// 使用 ShouldBind 配合错误处理
if err := c.ShouldBindJSON(&user); err != nil {
```



```
    HandleError(c, err)
    return
}
```

16.2. 📁📁📁📁

```
type Config struct {
    ServerPort string
    DatabaseURL string
}

func LoadConfig() *Config {
    return &Config{
        ServerPort: getEnv("SERVER_PORT", "8080"),
        DatabaseURL: getEnv("DATABASE_URL", "localhost:3306"),
    }
}
```

16.3. 📁📁📁📁

```
import (
    "log"
    "os"
    "time"

    "github.com/gin-gonic/gin"
)

func main() {
    // 自定义日志输出
    gin.DefaultWriter = os.Stdout
    gin.DefaultLogLevel = gin.Info

    r := gin.New()
    r.Use(gin.LoggerWithFormatter(func(param gin.LogFormatterParams) string
{
    return fmt.Sprintf("%s - [%s] \"%s %s %s %d %s\"\\n",
        param.ClientIP,
        param.TimeStamp.Format(time.RFC3339),
        param.Method,
        param.Path,
        param.Request.Proto,
        param.StatusCode,
        param.Latency,
    )
})))
    r.Use(gin.Recovery())
```

```
r.Run()
```

```
}
```

16.4. 🔒🔒🔒🔒🔒

```
package response

// 错误码定义
const (
    Success          = 0
    ServerError      = 10000
    BadRequest       = 10001
    Unauthorized     = 10002
    Forbidden        = 10003
    NotFound         = 10004
    ValidationError = 10005
)

var errorMsgs = map[int]string{
    Success:      "成功",
    ServerError:  "服务器内部错误",
    BadRequest:   "请求参数错误",
    Unauthorized: "未授权",
    Forbidden:    "禁止访问",
    NotFound:     "资源不存在",
    ValidationError: "验证失败",
}

type Response struct {
    Code    int    `json:"code"`
    Message string `json:"message"`
    Data    interface{} `json:"data,omitempty"`
}

func SuccessResponse(c *gin.Context, data interface{}) {
    c.JSON(http.StatusOK, Response{
        Code:    Success,
        Message: errorMsgs[Success],
        Data:    data,
    })
}

func ErrorResponse(c *gin.Context, code int) {
    c.JSON(http.StatusOK, Response{
        Code:    code,
        Message: errorMsgs[code],
    })
}
```

16.5. 🔒🔒🔒🔒🔒🔒 Viper🔒

```
go get github.com/spf13/viper
```

```
package config

import (
    "github.com/spf13/viper"
)

type Config struct {
    Server struct {
        Port int    `mapstructure:"port"`
        Mode string `mapstructure:"mode"`
    } `mapstructure:"server"`
    Database struct {
        Host      string `mapstructure:"host"`
        Port      int    `mapstructure:"port"`
        User      string `mapstructure:"user"`
        Password  string `mapstructure:"password"`
        DBName    string `mapstructure:"dbname"`
    } `mapstructure:"database"`
}

var AppConfig Config

func LoadConfig() error {
    viper.SetConfigName("config")
    viper.SetConfigType("yaml")
    viper.AddConfigPath(".")
    viper.AddConfigPath("./config")

    // 环境变量
    viper.SetConfigName("config")
    viper.SetConfigType("yaml")
    viper.AddConfigPath(".")
    viper.AddConfigPath("./config")

    // 环境变量
    viper.SetConfigName("config")
    viper.SetConfigType("yaml")
    viper.AddConfigPath(".")
    viper.AddConfigPath("./config")

    if err := viper.ReadInConfig(); err != nil {
        return err
    }

    return viper.Unmarshal(&AppConfig)
}
```

config.yaml:

```
server:
  port: 8080
  mode: debug

database:
  host: localhost
```

```
port: 3306
user: root
password: password
dbname: mydb
```

16.6. 数据库应用

```
package main

import (
    "github.com/gin-gonic/gin"
)

// 定义服务接口
type UserService interface {
    GetUser(id string) (*User, error)
    CreateUser(user *User) error
}

// 实现服务
type userServiceImpl struct {
    db *gorm.DB
}

func (s *userServiceImpl) GetUser(id string) (*User, error) {
    var user User
    err := s.db.First(&user, id).Error
    return &user, err
}

func (s *userServiceImpl) CreateUser(user *User) error {
    return s.db.Create(user).Error
}

// 控制器
type UserController struct {
    userService UserService
}

func NewUserController(service UserService) *UserController {
    return &UserController{userService: service}
}

func (ctrl *UserController) GetUser(c *gin.Context) {
    id := c.Param("id")
    user, err := ctrl.userService.GetUser(id)
    if err != nil {
        c.JSON(404, gin.H{"error": "User not found"})
        return
    }
    c.JSON(200, user)
}
```

```

}

func main() {
    r := gin.Default()

    // 初始化依赖
    db, _ := gorm.Open(...)
    userService := &userServiceImpl{db: db}
    userCtrl := NewUserController(userService)

    // 注册路由
    r.GET("/users/:id", userCtrl.GetUser)

    r.Run()
}

```

16.7. 自定义错误类型

```

package handler

import (
    "github.com/gin-gonic/gin"
)

// 错误处理中间件
func ErrorHandler() gin.HandlerFunc {
    return func(c *gin.Context) {
        defer func() {
            if err := recover(); err != nil {
                c.JSON(500, gin.H{
                    "code":    500,
                    "message":  "Internal Server Error",
                    "error":    err,
                })
                c.Abort()
            }
        }()
        c.Next()
    }
}

// 自定义错误类型
type AppError struct {
    Code    int
    Message string
}

func (e *AppError) Error() string {
    return e.Message
}

```

```
func NewAppError(code int, message string) *AppError {
    return &AppError{Code: code, Message: message}
}

// 使用示例
func SomeHandler(c *gin.Context) {
    user, err := getUser()
    if err != nil {
        // 业务错误
        if appErr, ok := err.(*AppError); ok {
            c.JSON(appErr.Code, gin.H{"error": appErr.Message})
            return
        }
        // 服务器错误
        c.JSON(500, gin.H{"error": "Internal Server Error"})
        return
    }
    c.JSON(200, user)
}
```

16.8. ❏❏❏❏

```
package main

import (
    "net/http"
    "time"

    "github.com/gin-gonic/gin"
)

func setupHealthCheck(r *gin.Engine) {
    // 简单健康检查
    r.GET("/health", func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{
            "status": "ok",
            "time":   time.Now().Format(time.RFC3339),
        })
    })

    // 详细健康检查
    r.GET("/health/detail", func(c *gin.Context) {
        status := gin.H{
            "status": "ok",
            "time":   time.Now().Format(time.RFC3339),
            "checks": gin.H{
                "database": checkDatabase(),
                "redis":     checkRedis(),
                "disk":       checkDisk(),
            },
        },
    })
}
```

```

        c.JSON(http.StatusOK, status)
    })
}

func checkDatabase() string {
    // 检查数据库连接
    sqlDB, err := db.DB()
    if err != nil {
        return "error: " + err.Error()
    }
    if err := sqlDB.Ping(); err != nil {
        return "error: " + err.Error()
    }
    return "ok"
}

func checkRedis() string {
    // 检查 Redis 连接
    _, err := rdb.Ping(ctx).Result()
    if err != nil {
        return "error: " + err.Error()
    }
    return "ok"
}

func checkDisk() string {
    // 检查磁盘空间
    var stat syscall.Statfs_t
    wd, _ := os.Getwd()
    syscall.Statfs(wd, &stat)
    available := stat.Bavail * uint64(stat.Bsize)
    availableGB := available / 1024 / 1024 / 1024
    if availableGB < 1 {
        return "warning: low disk space"
    }
    return "ok"
}

```

16.9. Docker 🗑️

Dockerfile:

```

# 构建阶段
FROM golang:1.21-alpine AS builder

WORKDIR /app

# 复制依赖文件
COPY go.mod go.sum ./
RUN go mod download

```

```
# 复制源代码
COPY . .

# 构建
RUN CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -o main .

# 运行阶段
FROM alpine:latest

RUN apk --no-cache add ca-certificates tzdata

WORKDIR /root/

# 从构建阶段复制二进制文件
COPY --from=builder /app/main .
COPY --from=builder /app/config ./config

# 设置时区
ENV TZ=Asia/Shanghai

EXPOSE 8080

CMD ["/main"]
```

docker-compose.yml:

```
version: '3.8'

services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      - GIN_MODE=release
      - DB_HOST=mysql
      - DB_PORT=3306
      - DB_USER=root
      - DB_PASSWORD=password
      - DB_NAME=mydb
    depends_on:
      - mysql
      - redis

  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: mydb
    volumes:
      - mysql_data:/var/lib/mysql
    ports:
```



```
    - "3306:3306"

  redis:
    image: redis:alpine
    ports:
      - "6379:6379"

  volumes:
    mysql_data:
```

16.10. Kubernetes ☒☒

deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gin-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: gin-app
  template:
    metadata:
      labels:
        app: gin-app
    spec:
      containers:
        - name: gin-app
          image: myregistry/gin-app:latest
          ports:
            - containerPort: 8080
          env:
            - name: GIN_MODE
              value: "release"
            - name: DB_HOST
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: host
      resources:
        requests:
          memory: "128Mi"
          cpu: "250m"
        limits:
          memory: "256Mi"
          cpu: "500m"
      livenessProbe:
        httpGet:
          path: /health
```

```

    port: 8080
    initialDelaySeconds: 10
    periodSeconds: 30
  readinessProbe:
    httpGet:
      path: /health
      port: 8080
    initialDelaySeconds: 5
    periodSeconds: 10

```

service.yaml:

```

apiVersion: v1
kind: Service
metadata:
  name: gin-app-service
spec:
  selector:
    app: gin-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: LoadBalancer

```

16.11. 🔒🔒🔒🔒

```

go get github.com/prometheus/client_golang/prometheus
go get github.com/prometheus/client_golang/prometheus/promhttp

```

```

package main

import (
    "github.com/gin-gonic/gin"
    "github.com/prometheus/client_golang/prometheus"
    "github.com/prometheus/client_golang/prometheus/promhttp"
)

var (
    httpRequestsTotal = prometheus.NewCounterVec(
        prometheus.CounterOpts{
            Name: "http_requests_total",
            Help: "Total number of HTTP requests",
        },
        []string{"method", "path", "status"},
    )

```

```

    httpRequestDuration = prometheus.NewHistogramVec(
        prometheus.HistogramOpts{
            Name:    "http_request_duration_seconds",
            Help:    "HTTP request duration in seconds",
            Buckets: prometheus.DefBuckets,
        },
        []string{"method", "path"},
    )
}

func init() {
    prometheus.MustRegister(httpRequestsTotal)
    prometheus.MustRegister(httpRequestDuration)
}

func PrometheusMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        start := time.Now()

        c.Next()

        duration := time.Since(start).Seconds()
        status := strconv.Itoa(c.Writer.Status())

        httpRequestsTotal.WithLabelValues(c.Request.Method, c.FullPath(),
            status).Inc()
        httpRequestDuration.WithLabelValues(c.Request.Method,
            c.FullPath()).Observe(duration)
    }
}

func main() {
    r := gin.New()
    r.Use(PrometheusMiddleware())

    // Prometheus 指标端点
    r.GET("/metrics", gin.WrapH(promhttp.Handler()))

    r.GET("/api", func(c *gin.Context) {
        c.JSON(200, gin.H{"message": "OK"})
    })

    r.Run()
}

```

17. 性能优化

17.1. 性能优化

```
func main() {
    r := gin.Default()

    // 方式1: 默认监听 0.0.0.0:8080
    r.Run()

    // 方式2: 自定义地址
    r.Run(":8081")

    // 方式3: 带 TLS (HTTPS)
    r.RunTLS(":443", "server.crt", "server.key")

    // 方式4: 使用 HTTP/2
    // r.RunTLS(":443", "server.crt", "server.key")

    // 方式5: 使用自定义 HTTP 服务器 (推荐生产环境)
    s := &http.Server{
        Addr:           ":8080",
        Handler:         r,
        ReadTimeout:    10 * time.Second,
        WriteTimeout:   10 * time.Second,
        MaxHeaderBytes: 1 << 20,
    }
    s.ListenAndServe()
}
```

17.2. 使用 net/http 包

```
import (
    "os"
    "strconv"
)

func main() {
    port := getEnv("PORT", "8080")

    r := gin.Default()
    r.Run(":" + port)
}

func getEnv(key, defaultValue string) string {
    if value := os.Getenv(key); value != "" {
        return value
    }
    return defaultValue
}

func getEnvInt(key string, defaultValue int) int {
    if value := os.Getenv(key); value != "" {
        if intVal, err := strconv.Atoi(value); err == nil {
            return intVal
        }
    }
    return defaultValue
}
```

```

        return intVal
    }
}
return defaultValue
}

```

17.3. ☒☒☒☒☒☒

```

import (
    "os"
    "time"

    "github.com/gin-gonic/gin"
)

func main() {
    // 创建日志文件
    logFile, _ := os.Create("/var/log/gin/" + time.Now().Format("2006-01-02") + ".log")

    gin.SetMode(gin.ReleaseMode)
    r := gin.New()

    // 使用文件输出日志
    gin.DefaultWriter = logFile
    gin.DefaultErrorWriter = logFile

    r.Use(gin.Logger())
    r.Use(gin.Recovery())

    r.GET("/ping", func(c *gin.Context) {
        c.JSON(200, gin.H{"message": "pong"})
    })

    r.Run()
}

```

17.4. ☒☒☒☒

```

import (
    "log"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/gin-gonic/gin"
)

```

```
func main() {
    r := gin.Default()

    s := &http.Server{
        Addr:    ":8080",
        Handler: r,
    }

    // 启动服务器
    go func() {
        if err := s.ListenAndServe(); err != nil && err !=
http.ErrServerClosed {
            log.Fatalf("Server error: %v", err)
        }
    }()

    // 等待中断信号
    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
    <-quit

    log.Println("Shutting down server...")

    // 给予 5 秒完成剩余请求
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    if err := s.Shutdown(ctx); err != nil {
        log.Fatal("Server forced to shutdown:", err)
    }

    log.Println("Server exited")
}
```

17.5. 🔒🔒🔒🔒

```
# 使用 air 实现热重载（开发环境）
go get -u github.com/cosmtrek/air

# 创建 .air.toml 配置文件
# 运行
air
```

```
# 使用 nginx 实现负载均衡（生产环境）
upstream backend {
    server 127.0.0.1:8080;
    server 127.0.0.1:8081;
}
```

```
server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

18. 📖

📖 Gin 📖

18.1. 📖

1. 📖Go 📖Gin 📖
2. 📖Gin 📖
3. 📖

18.2. 📖

4. 📖JSON/Form/URI/Header 📖
5. 📖Context 📖/📖
6. 📖/📖

18.3. 📖

7. 📖/📖
8. 📖GORM 📖CRUD📖
9. 📖
10. 📖Basic Auth📖JWT📖CORS📖

18.4. 📖

11. 📖WebSocket📖SSE📖Swagger📖
12. 📖gzip 📖

18.5. 📖

13. 📖Docker/K8s 📖
14. 📖HTTPS 📖

18.6. 📖

18.6.1. 📖

```
# 安装 Gin
go get -u github.com/gin-gonic/gin

# 安装 GORM
go get -u gorm.io/gorm
go get -u gorm.io/driver/mysql

# 安装常用中间件
go get -u github.com/gin-contrib/cors
go get -u github.com/golang-jwt/jwt/v5
go get -u github.com/swaggo/swag/cmd/swag

# 生成 Swagger 文档
swag init

# 热重载开发
go get -u github.com/cosmtrek/air
air
```

18.6.2. XXXXX

XX	XX	XX
ORM	gorm.io/gorm	XXX ORM
XX	github.com/spf13/viper	XXXX
XX	go.uber.org/zap	XXXXX
XX	github.com/go-playground/validator/v10	XXXX
JWT	github.com/golang-jwt/jwt/v5	JWT XX
CORS	github.com/gin-contrib/cors	CORS XXX
WebSocket	github.com/gorilla/websocket	WebSocket XX
Redis	github.com/go-redis/redis/v8	Redis XXX
XX	github.com/swaggo/gin-swagger	Swagger XX
XX	github.com/prometheus/client_golang	Prometheus XX
XXXXX	github.com/robfig/cron/v3	Cron XXXX
XX	gopkg.in/gomail.v2	XXXX
UUID	github.com/google/uuid	UUID XX